

You are right to be frustrated. My apologies. The previous solution failed because even with a Platform.OS check, the line `import LoaderKit from "react-native-loader-kit";` at the top of the file forces Metro to try and bundle that library for **all platforms**, which causes the web build to fail *before* your runtime code is ever executed.

This is a build-time error, not a runtime error.

The correct and most robust solution is to use **platform-specific file extensions**. This tells the Metro bundler to use a completely different file for web than it does for native, preventing the problematic import from ever entering the web bundle.

Here is the step-by-step solution.

---

## Step 1: Create a Web-Safe Loader Component

Create a new file in your components directory called `PlatformLoader.web.tsx`. This file will **only** be used for the web build.

**File: `src/components/PlatformLoader.web.tsx`**

```
code TypeScript
downloadcontent_copy
expand_less
import { ActivityIndicator } from 'react-native';

// This is the web-safe version of our loader.
// It deliberately does NOT import react-native-loader-kit.
export const PlatformLoader = () => {
  return <ActivityIndicator color="#fff" />;
};
```

---

## Step 2: Create a Native-Only Loader Component

Create another new file with the *exact same name* but a `.native.tsx` extension. This will be used for iOS and Android.

**File: `src/components/PlatformLoader.native.tsx`**

```
code TypeScript
downloadcontent_copy
expand_less
IGNORE_WHEN_COPYING_START
IGNORE_WHEN_COPYING_END
import LoaderKit from 'react-native-loader-kit';
```

```
// This version is only for native and can safely import LoaderKit.
export const PlatformLoader = () => {
  return <LoaderKit name="LineScaleParty" color="#fff" />;
};
```

---

### Step 3: Update Your ListItem Component

Now, you can clean up your ListItem component. You will import PlatformLoader without any file extension. Metro will automatically pick the correct file based on the platform you're bundling for.

**Here is your final, corrected ListItem.tsx file:**

```
code JavaScript
downloadcontent_copy
expand_less
IGNORE_WHEN_COPYING_START
IGNORE_WHEN_COPYING_END
import { unknownTrackImageUri } from "@constants/images";
import { TouchableHighlight, View, Image, Text } from "react-native";
import { useThemeColor } from '@/hooks/useThemeColor';
import { Track, useActiveTrack, useIsPlaying } from "react-native-track-
player";
import { Entypo, Ionicons } from "@expo/vector-icons";

// Import the new platform-specific component
import { PlatformLoader } from './PlatformLoader'; // Adjust path if needed

export type ListItemProps = {
  track: Track;
  onTrackSelect: (track: Track) => void;
}

export const ListItem = ({ track, onTrackSelect: handleTrackSelect }:
ListItemProps) => {
  const isActiveTrack = useActiveTrack()?.url === track.url;
  const { playing } = useIsPlaying();

  const textColor = useThemeColor({}, 'text');
  const mutedTextColor = useThemeColor({}, 'textMuted');

  return (
    <TouchableHighlight onPress={() => handleTrackSelect(track)}
underlayColor="rgba(0,0,0,0.1)">
      <View className="flex-row items-center gap-x-3.5 px-4 py-2">
        <View className="relative h-[50px] w-[50px] items-center
justify-center">
          <Image
            source={{ uri: track.image ?? unknownTrackImageUri }}
            className={`h-full w-full rounded-lg ${`
```

```

            isActiveTrack ? 'opacity-60' : 'opacity-100'
        }` }
    />
    {isActiveTrack && (
        <View className="absolute inset-0 flex items-center
justify-center rounded-lg bg-black/40">
            {playing ? (
                // Use the new component. No conditional
logic needed here.
                <PlatformLoader />
            ) : (
                <Ionicons name="play" size={24} color="#fff"
/>
            )}
        </View>
    )}
</View>

<View className="flex-1">
    <Text
        numberOfLines={1}
        className={`text-sm max-w-[90%] ${
            isActiveTrack ? 'font-semibold' : 'font-normal'
        }`}
        style={{ color: isActiveTrack ? textColor :
mutedTextColor }}
    >
        {track.title}
    </Text>
    {track.artist && (
        <Text
            className="mt-1 text-xs"
            style={{ color: mutedTextColor }}
        >
            {track.artist}
        </Text>
    )}
</View>

    <Entypo name="dots-three-horizontal" size={18}
color={mutedTextColor} />
</View>
</TouchableHighlight>
    );
}

```

## Step 4: Full Reset (Crucial)

Metro can cache failed module resolutions. To ensure this fix works, you must do a full reset.

1. **Stop the Metro server (Ctrl+C).**
2. **Delete the cache folder: `rm -rf .expo`**
3. **Restart with the clear flag:**

code Bash

downloadcontent\_copy

expand\_less

IGNORE\_WHEN\_COPYING\_START

IGNORE\_WHEN\_COPYING\_END

```
npx expo start --clear
```

This approach completely isolates the native-only code from the web bundle, which will permanently solve this category of error.